

NAME - KRISH CHANDRASHEKHAR PACHODE ROLL NO.- SY-B-18 BATCH - AIDS(SY-B)

THEORY -

1. What is web scraping? Explain the role of the Requests library in web scraping.**

Web scraping is the process of extracting data from websites automatically using Python programs. It helps in collecting useful information such as text, images, or links from web pages.

Role of Requests library: The Requests library is used to send HTTP requests to websites and retrieve their HTML content. It acts as a connection between the Python program and the web server.

Example:

```
import requests
response = requests.get("https://example.com")
html = response.text
```

2. What is BeautifulSoup? How does it help in parsing HTML?**

BeautifulSoup is a Python library used to parse HTML and XML documents. It converts raw HTML into a structured format (parse tree), making it easy to extract required data.

Uses:

- Navigates HTML elements
- Extracts text and attributes
- Searches specific tags

Example:

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
```

3. Differentiate between web scraping and web crawling.**

Feature	Web Scraping	Web Crawling
Purpose	Extract specific data	Discover and index web pages
Scope	Limited to specific site	Covers multiple websites
Example	Getting product prices	Google search engine crawling

Scraping is data extraction, while crawling is page discovery.

4. What are the commonly used methods in BeautifulSoup?**

- `find()` → Returns first matching element
- `find_all()` → Returns all matching elements
- `get_text()` → Extracts text from HTML tags
- `get()` → Retrieves attribute values (like links)

- `select()` → Uses CSS selectors to find elements

Example:

```
soup.find("h1")
soup.find_all("p")
```

5. What are the ethical and legal considerations in web scraping?**

- Follow website rules (robots.txt)
- Do not send too many requests (avoid server overload)
- Avoid scraping private or sensitive data
- Respect copyright and terms of service
- Use data responsibly

Ethical scraping ensures legal and fair use of web data.

```
# -----
# Assignment 6: Web Scraping using BeautifulSoup
# -----

# Step 1: Import required libraries
import requests
from bs4 import BeautifulSoup
import csv

# Step 2: Send HTTP request
url = "https://example.com"
response = requests.get(url)

# Step 3: Parse HTML content
soup = BeautifulSoup(response.text, "html.parser")

# -----
# Title Extraction
# -----
print("\n----- PAGE TITLE -----")
print(soup.title.text)

# -----
# Paragraph Extraction
# -----
print("\n----- PARAGRAPHS -----")
paragraphs = soup.find_all("p")

for p in paragraphs:
    print(p.text)

# -----
# Headings Extraction
# -----
print("\n----- HEADINGS (H1, H2) -----")
headings = soup.find_all(["h1", "h2"])
```

```

for h in headings:
    print(h.text)

# -----
# Links Extraction
# -----
print("\n----- LINKS -----")
links = soup.find_all("a")

all_links = []
for link in links:
    href = link.get("href")
    print(href)
    all_links.append(href)

# -----
# Save Links to CSV
# -----
with open("links.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Links"])

    for link in all_links:
        writer.writerow([link])

print("\nLinks saved to CSV file")

# -----
# Generalized Scraper (Advanced)
# -----
print("\n----- GENERALIZED SCRAPER -----")

tag = input("Enter tag to search (e.g., p, a, h1): ")

elements = soup.find_all(tag)

print(f"\nExtracted {tag} elements:")
for el in elements:
    print(el.get_text(strip=True))

# -----
# End of Program
# -----
print("\nWeb scraping completed successfully.")

```

```

----- PAGE TITLE -----
Example Domain

```

```

----- PARAGRAPHS -----
This domain is for use in documentation examples without needing permission. Avoid use in operations.
Learn more

```

```

----- HEADINGS (H1, H2) -----
Example Domain

```

```
----- LINKS -----  
https://iana.org/domains/example  
  
Links saved to CSV file  
  
----- GENERALIZED SCRAPER -----  
Enter tag to search (e.g., p, a, h1): P  
  
Extracted P elements:  
  
Web scraping completed successfully.
```

Conclusion

In this assignment, web scraping was successfully implemented using Python with the help of the Requests and BeautifulSoup libraries. The requests library was used to fetch webpage content, while BeautifulSoup was used to parse and extract useful information from HTML.

Different elements such as titles, paragraphs, headings, and links were extracted from the webpage. The process of web scraping was performed step by step, including sending requests, parsing HTML, finding elements, and extracting data.

Additionally, the extracted data was stored in a CSV file, demonstrating how scraped data can be saved for further use. Ethical considerations were also discussed to ensure responsible use of web scraping techniques.

Overall, this assignment provided a clear understanding of how automated data extraction works and how it can be applied in real-world scenarios such as data analysis, research, and automation.